AGENTIC AI TRAVEL PLANNER

CODEX-READY BACKEND IMPLEMENTATION BLUEPRINT

A self-contained handoff document: exact files, ownership, APIs, data contracts, agent workflow, security rules, test expectations and copy-paste Codex prompts.

Stack: Python 3.12 | FastAPI | LangGraph | LangChain | RAG | PostgreSQL + pgvector | Redis | Celery | Docker | AWS

Status: Implementation specification, not already-written or tested code. The 1 Nov 2026 release target is aggressive; prioritize a working vertical slice and do not label mock quotes as live fares.

A. Product scope and non-negotiable decisions

• ChatGPT-inspired web interface; guest mode + optional account signup/login and profile.

• Two chat modes ONLY: Temporary = tab-scoped, no permanent chat history; Saved = authenticated, fixed expiry 7 days after creation. No shared chat feature.

• Password recovery uses email six-digit OTP -> verify OTP -> short-lived reset authorization -> new password. No reset links.

• Multi-city travel: natural language, flexible city ordering, deadlines, buses/trains/flights/roads, hotels, places, entry charges, budget, maps and booking redirects.

• Frontend renders graphical donut/pie/bar charts from backend JSON; backend does not produce chart images.

• Prices must carry source, freshness, price type (live_quote / estimate / unknown); no invented seat or hotel availability.

• Backend-first delivery, but contract tests must guarantee frontend compatibility.

B. Request and agent flow

```
Browser -> FastAPI API -> Auth/guest + input validation + rate limit
       -> ChatService -> enqueue TripJob -> Celery worker
       -> LangGraph: intent -> research -> plan -> validate -> respond
                      |           |         ^          |
                      |           +-- provider APIs & RAG  |
                      +------ if missing fields: ask user  |
                                  bounded repair <---------+
       -> typed TripResponse -> save only if saved chat
       -> REST retrieval + SSE progress -> React cards/charts
```

Technical boundary: LLM extracts preferences and explains; deterministic route-time/cost arithmetic and source-backed availability determine feasibility. Max planning retries: 2. Missing live data remains unknown, never silently assumed.

C. Exact backend folder structure and file responsibilities

Each line defines one implementation file and its purpose. Create files only when implementing their milestone. __init__.py files can be added as needed.

Entry & API

app/main.py — FastAPI app, lifecycle, middleware, routers, exception handlers.

app/api/router.py — Aggregate and version API routers.

app/api/v1/health.py — GET /health and /ready; readiness checks dependencies.

app/api/v1/auth.py — Signup, login, logout, refresh/current session, password reset finish.

app/api/v1/otp.py — Forgot password OTP request, verify and resend endpoints.

app/api/v1/users.py — GET/PATCH me, change password, account deletion.

app/api/v1/conversations.py — Create/list/get/delete saved conversations, 7-day filter.

app/api/v1/messages.py — Save message and start/retrieve chat execution.

app/api/v1/temporary_chat.py — Guest or logged-in temporary chat endpoint without history.

app/api/v1/trips.py — Get planning outputs, itinerary versions and quote metadata.

app/api/v1/streaming.py — SSE planning status; session-bound authorization.

Core & database

app/core/config.py — Typed env settings and feature flags; never expose secrets.

app/core/security.py — Argon2id password hash, opaque session cookie/token checks, CSRF strategy.

app/core/dependencies.py — FastAPI dependency injection for database, Redis and identity.

app/core/rate_limiter.py — Per-account/IP/session limits for login, OTP, agents.

app/core/exceptions.py — Consistent public-safe error structure.

app/core/logging.py — Structured logs with sensitive data redaction.

app/core/constants.py — Retention, quote status and processing constants.

app/db/base.py — SQLAlchemy declarative base, model imports.

app/db/session.py — Async engine, pooled sessions and transaction scope.

app/db/checkpointer.py — LangGraph checkpoint persistence and deletion strategy.

Database models

app/models/user.py — Account name/email/hashed password, timestamps.

app/models/auth_session.py — Opaque session hash, user FK, expiry, revocation.

app/models/password_reset_otp.py — OTP HMAC, attempts, cooldown, expiry, reset authorization.

app/models/conversation.py — Saved-only conversation, owner, title, fixed expires_at.

app/models/message.py — Role, content, conversation FK, timestamps.

app/models/trip.py — Trip requirements, job status, source provenance.

app/models/itinerary.py — Versioned itinerary JSON and FK to trip/conversation.

app/models/knowledge_document.py — RAG content/source/update timestamp/vector.

Pydantic schemas

app/schemas/auth.py — Signup/login, session and password change DTOs.

app/schemas/otp.py — Request/verify/resend/reset DTOs, constrained code.

app/schemas/user.py — Profile DTO without password hashes.

app/schemas/conversation.py — Saved chat lifecycle and expiry metadata DTO.

app/schemas/message.py — User/assistant message DTOs.

app/schemas/trip.py — Prompt, origin, stops, dates, constraints, results.

app/schemas/itinerary.py — Day/timeslot/action with time zone fields.

app/schemas/transport.py — Flight/train/bus/road normalized provider offer.

app/schemas/hotel.py — Stay options, availability and quote provenance.

app/schemas/attraction.py — Places, fee, opening hours, freshness and crowd caveat.

app/schemas/budget.py — INR totals, itemized estimate and chart data.

app/schemas/common.py — Currency, source metadata, errors and pagination.

Repositories

app/repositories/user_repository.py — User CRUD by id/email.

app/repositories/session_repository.py — Create/get/revoke hashed sessions.

app/repositories/otp_repository.py — Atomic OTP attempts and token consumption.

app/repositories/conversation_repository.py — Owner-scoped chats; enforce expires_at queries.

app/repositories/message_repository.py — Message persistence and pagination.

app/repositories/trip_repository.py — Trip status, job ownership and version queries.

app/repositories/itinerary_repository.py — Itinerary versions and associated cleanup.

Business services

app/services/auth_service.py — Signup, login, logout and secure session policies.

app/services/otp_service.py — CSPRNG OTP, HMAC, verify, invalidation, reset grant.

app/services/email_service.py — AWS SES or configured email provider adapter.

app/services/user_service.py — Profile changes and account deletion.

app/services/conversation_service.py — Saved chat ownership, lifecycle and fixed 7-day TTL.

app/services/temporary_chat_service.py — Tab-scoped ephemeral chat context; Redis TTL.

app/services/chat_service.py — Dispatch message and job, format chat results.

app/services/trip_service.py — Trip job lifecycle and stable output versioning.

app/services/route_service.py — Routes, legs, candidate order and time zone accuracy.

app/services/transport_service.py — Compare multi-modal options, provider normalization.

app/services/hotel_service.py — Hotel discovery, provenance and budget filter.

app/services/attraction_service.py — Sightseeing search, time slots, fee evidence.

app/services/budget_service.py — Deterministic sums and frontend chart datasets.

app/services/itinerary_service.py — Time-stamped plan construction and checks.

app/services/booking_service.py — Allowlisted provider URLs; no confirmed booking claims.

app/services/cleanup_service.py — Delete expired saved chats, messages, plans and checkpoints.

LangGraph agents

app/agents/state.py — Typed workflow state; requirements, research, plan, errors.

app/agents/graph.py — Compile workflow and checkpointing.

app/agents/llm.py — Model configuration, output schema and limits.

app/agents/prompts.py — Intent/research/planning/response prompt templates.

app/agents/edges.py — Conditional edges, clarify flow, bounded retries.

app/agents/nodes/intent_node.py — Prompt -> strict TripRequirements and ambiguities.

app/agents/nodes/research_node.py — Tool calls, retrieval and provenance normalization.

app/agents/nodes/planning_node.py — Candidate itinerary assembly from researched options.

app/agents/nodes/validation_node.py — Deterministic deadline/budget/route validation.

app/agents/nodes/response_node.py — Final typed response including warnings and chart data.

Travel integrations & RAG

app/integrations/base.py — Provider interface, timeout, retry, cache and error types.

app/integrations/maps_provider.py — Geocoding, directions, travel duration.

app/integrations/places_provider.py — Places metadata and hours; attribution.

app/integrations/flight_provider.py — Authorized flight search adapter; optional capability.

app/integrations/train_provider.py — Authorized train search adapter; avoid unsupported seat claims.

app/integrations/bus_provider.py — Authorized bus schedule/quote adapter.

app/integrations/hotel_provider.py — Authorized hotel inventory or search redirect.

app/integrations/booking_provider.py — Verify known provider booking/deep links.

app/rag/ingestion.py — Read vetted travel documents and track provenance.

app/rag/embeddings.py — Embedding model configuration.

app/rag/vector_store.py — pgvector table/collection setup.

app/rag/retriever.py — Retrieve relevant travel knowledge with sources.

Tasks, utilities, infrastructure and tests

app/tasks/celery_app.py — Celery worker and broker configuration.

app/tasks/trip_tasks.py — Execute AI runs, progress statuses and failures.

app/tasks/cleanup_tasks.py — Scheduled cleanup of expired data and temp state.

app/utils/datetime_utils.py — Time zones, dates, durations and office deadline checks.

app/utils/currency_utils.py — Money arithmetic via Decimal and currency formatting.

app/utils/url_validator.py — Allowlisted external booking links and SSRF defenses.

alembic/env.py — Migration environment and model metadata.

tests/conftest.py — Isolated test DB and fake provider fixtures.

tests/test_auth.py — Signup/login/ownership/CSRF/session protections.

tests/test_otp.py — Expiry, resend, attempts, reset grant replay tests.

tests/test_conversations.py — 7-day expiry, authorization, history list.

tests/test_temporary_chat.py — No persistence and cross-tab/session isolation.

tests/test_chat_cleanup.py — Delete messages, trips, checkpoints, traces.

tests/test_trip_planning.py — Request -> structured validated itinerary.

tests/test_route_optimization.py — Date conflict, city order, connection feasibility.

tests/test_budget.py — Decimal sums, unknown-vs-estimate price labels.

tests/test_langgraph.py — State transitions, bounded retries and failure paths.

tests/test_api_security.py — Rate limits, injection and booking URL safeguards.

scripts/init_db.py — Dev DB setup helper; migrations are authoritative.

scripts/ingest_knowledge.py — RAG ingestion CLI.

Dockerfile — Build API/worker from one reproducible image.

pyproject.toml — Locked or constrained dependencies, lint/test config.

.env.example — All variables with safe placeholders.

README.md — Startup, migrations, API tests and architecture notes.

D. API contract baseline

Method / route	Expected behavior
POST /api/v1/auth/signup	Create account with hashed password
POST /api/v1/auth/login; /logout	Set/revoke secure HttpOnly cookie
POST /api/v1/auth/forgot-password	Generic response; email 6-digit OTP
POST /api/v1/auth/verify-otp	Verify and issue one-time reset grant
POST /api/v1/auth/resend-otp	Cooldown and invalidate old code
POST /api/v1/auth/reset-password	Grant + new password -> revoke all sessions
GET /api/v1/users/me	Authenticated profile
POST /api/v1/conversations	Create saved conversation for signed-in user
GET /api/v1/conversations	Only owner and non-expired chats
DELETE /api/v1/conversations/{id}	Owner deletion with dependent cleanup
POST /api/v1/conversations/{id}/messages	Start planning job from saved chat
POST /api/v1/chat/temporary	Guest temporary message; no durable history
GET /api/v1/trips/{id}	Ownership/session check, typed trip result
GET /api/v1/trips/{id}/events	SSE progress/status (protect access)
GET /api/v1/health; /ready	Liveness vs external dependency readiness

E. Typed response contract (frontend and Recharts)

```
{
  "trip_id": "opaque_id", "status": "completed",
  "requirements": {"origin":"Hyderabad","destinations":["Puducherry","Rameswaram"],
    "start_date":"2026-10-30","end_date":"2026-11-01",
    "return_deadline":"2026-11-02T10:00:00+05:30","currency":"INR"},
  "feasible": false, "warnings":["Illustrative; verify connections"],
  "route_legs": [], "transport_options": [], "hotels": [],
  "attractions": [], "itinerary_days": [],
  "budget": {"currency":"INR","total_estimated":null,
    "categories":[], "price_type":"unknown"},
  "data_sources": [], "booking_links": []
}
```

Response values above are schema examples, not an actual priced itinerary. Use null when price is unknown; each offer includes origin, destination, provider name, fetched_at, fare type, verified booking URL and availability status.

F. Chat lifecycle and data policy

• Temporary: React tab memory + optional Redis transient execution context; no saved conversation row, no sidebar, no localStorage. Tab closure removes client access, server deletion depends on TTL and cleanup.

• Saved: only signed-in users; expires_at = created_at + 7 days, never extended on activity; every read enforces owner + expiry.

• Cleanup: delete conversation, messages, trips, itinerary versions and LangGraph checkpoints, and avoid retaining prompts in application logs.

• Backup caveat: deletion from active DB is not instant permanent erasure from retained backups; define backup purge/expiry before making strong deletion claims.

• User can manually delete own chat, logout and remove account. No share links or sharing APIs.

G. OTP verification (no email reset links)

```
Email -> issue 6-digit OTP (CSPRNG) -> email delivery
    -> verify 5-min OTP (<=5 attempts, resend cooldown 60s)
    -> consume OTP -> issue one-time, short-lived reset grant
    -> new password + reset grant -> hash password -> revoke sessions
```

Use constant/generic responses for unknown emails, per-IP and per-identity limits, no OTP/password in logs, secure random and keyed OTP hash. Reset grant must be server-validated; frontend verified=true is not sufficient.

H. Acceptance tests before calling backend complete

• /health works; /ready fails appropriately when Redis or PostgreSQL unavailable.

• Signup, login, logout, wrong password, protected profile, expired/revoked cookie tested.

• Unknown email OTP request does not reveal account existence; brute-force, expiry, resend and reuse tests pass.

• Two users cannot read one another’s chats or trips; expired chat inaccessible even before cleanup.

• Temporary messages never write to permanent saved conversation history.

• Trip prompt generates typed itinerary or clearly states unfeasibility, never fabricates live inventory.

• Budget chart figures equal total and preserve unknown/estimated provenance.

• Job timeout and API outage surface graceful errors; progress stream stops after completion.

• Docker start, migrations, lint, unit tests and API smoke test run cleanly.

I. Codex execution playbook — use one module at a time

Use these exact prompts in Codex. Before running, paste/upload this PDF and ask Codex to read it in full. If Codex cannot read the PDF directly, give it the extracted text in a backend_spec.md file.

Prompt 0 — Read and plan

Read the attached Backend Implementation Blueprint in full. Inspect the repository. Do NOT generate all files at once. Summarize the planned architecture, dependencies, milestones and the first smallest end-to-end working slice. Identify ambiguous contracts before writing code. Explain in simple Hinglish.

Prompt 1 — Foundation

Implement only app/main.py, config, async DB + Redis, routing, /health and /ready, Docker and test harness. Use Python 3.12 / FastAPI / SQLAlchemy 2 / Pydantic v2. Inspect repo first; preserve existing code. Add runnable tests. Explain every modified file, important function and commands in Hinglish. Run tests and report results.

Prompt 2 — Authentication + Email OTP

Implement database-backed secure account sessions and signup/login/logout, profile basics, six-digit email OTP password reset (no email reset link). Five-minute expiry, five attempts, 60-second resend cooldown, one-time reset grant, session revocation. Add migrations and tests for enumeration, replay and rate limits. Never log secrets.

Prompt 3 — Chat retention

Implement saved conversations for authenticated users with fixed 7-day expiry, per-user ownership, messages, manual deletion and cleanup job. Temporary mode must not create saved history; use Redis TTL only if necessary. Include LangGraph checkpoints cleanup plan, tests for expiry, ownership and tab behavior.

Prompt 4 — LangGraph vertical slice

Create typed TravelState and five nodes: intent, research, planning, validation, response. Begin with clearly labelled stub providers; connect a real route/maps API only when configured. Return strict TripResponse JSON and bounded retries. Date and budget checks must be deterministic. Add graph tests.

Prompt 5 — Live providers + RAG

Implement provider interfaces and adapters for maps, places, train, bus, flight, hotels. Use authorized providers only. For unavailable provider APIs, return unsupported/unknown with safe provider discovery link, not fabricated prices. Add small source-dated RAG knowledge base and provenance in responses.

Prompt 6 — Background jobs + integration

Add Celery worker + Redis broker, plan job statuses, SSE progress and frontend-friendly API responses. Add cleanup scheduler, source freshness, logging redaction, timeouts, ownership checks and e2e tests. Keep existing endpoints stable.

Prompt 7 — Teach + review

Explain all changes file-by-file in easy Hinglish: purpose, inputs/outputs, data flow, security decisions, test procedure, failure cases. Do not edit code during explanation. Then perform review and report critical/high/medium issues and exact commands actually executed.

J. Practical implementation order and release gate

Priority	Must be working
P0	Working request -> LangGraph -> typed TripResponse -> frontend-ready JSON
P0	Secure auth + OTP, no account enumeration
P0	Saved 7-day chats and truly temporary history behavior
P0	Deadline feasibility, data provenance, stable error responses
P1	Live provider coverage where approved APIs exist; RAG; richer hotel data
P1	AWS managed deployment and scaling; multi-provider retries

Do not postpone all integration until the last day. Even if frontend visuals are built later, keep one thin request/response contract smoke test running from day one.